Practices for Git Collaboration with Multiple Contributors (Common Issues and Resolutions)

==========

1. Set up a clear workflow:

- Use the following branch structure:
  - `main` → Main branch for stable, working code.
  - `feature/*` → Branches for individual changes/features.
  - `fix/*` → Branches for bug fixes.

2. Use a consistent update process before modifying files:

Before making any changes:

```
git pull origin main --rebase
```

- `--rebase` keeps the commit history cleaner by placing your changes on top of the latest updates from the remote repository.

3. Merge conflicts during pull or push

Problem: You modify a file that has also been changed by someone else.
Resolution:

1. Fetch and rebase before committing:
   ```
   git fetch origin
   git rebase origin/main
   ```

2. If conflicts appear:
   - Open the conflicting file(s) → Look for `<<<<<<<`, `=======`, `>>>>>>>` markers.
   - Resolve conflicts manually.
   - After resolving conflicts:

   ```
   git add .
   git rebase --continue
   ```

4. Error: “Cannot fast-forward”

Problem: Your local branch is behind the remote branch.
Resolution:

- Fetch the latest changes and rebase:
  ```
  git fetch origin
  git rebase origin/main
  ```

- If that fails due to conflicts:
  ```
  git merge origin/main
  ```

5. Error: “Push rejected due to remote changes”

Problem: The remote branch has new commits that conflict with your local changes.
Resolution:

- First, pull the latest changes using `rebase`:
  ```
  git pull --rebase origin main
  ```

- If you get conflicts, fix them manually and continue the rebase:
  ```
  git add .
  ```

```
git rebase --continue
```

- Finally, push:
```
git push origin main
```

6. Error: “Detached HEAD state”

Problem: You checked out a specific commit instead of a branch.
Resolution:

1. Create a new branch from the detached state:
```
git checkout -b new-branch
```

2. Merge it back to `main` after testing:
```
git checkout main
git merge new-branch
git push origin main
```

7. Local changes conflict with remote after push

Problem: Another contributor has pushed changes before you pushed yours.
Resolution:

1. Use `rebase` to align your work with the latest remote changes:
```
git pull --rebase origin main
```

2. Resolve conflicts if needed.
3. Push the merged code:
```
git push origin main
```

8. Error: “Updates were rejected because the tip of your current branch is behind”

Cause: The remote repository has changes that you don't have locally.
Solution:

1. First, pull the latest changes:
```
git pull --rebase origin main
```

2. If conflicts occur, resolve them manually.
3. After resolving conflicts:
```
git add .
git rebase --continue
```
4. Then push:
```
git push origin main
```

9. Error: “fatal: remote upstream already exists”

Cause: You may have already added an upstream remote.
Solution:

1. Check the remotes:
```
git remote -v
```
2. If `upstream` already exists, remove it:
```
git remote remove upstream
```
3. Add it again (if needed):
```
git remote add upstream
https://github.com/original-owner/repo.git
```

10. Error: “Conflicts during pull”

Cause: Your local branch and remote branch have diverged.
Solution:

1. Stash your changes:
   ```
   git stash
   ```
2. Pull the latest changes:
   ```
   git pull --rebase origin main
   ```
3. Apply your stashed changes:
   ```
   git stash pop
   ```
4. If conflicts appear, fix them manually and continue:
   ```
   git add .
   git rebase --continue
   ```

11. Error: “Nothing to commit”

Cause: No changes are staged for commit.
Solution:

1. Check the current status:
   ```
   git status
   ```
2. If files are modified but not staged:
   ```
   git add .
   git commit -m "Update message"
   ```
3. Push the commit:
   ```
   git push origin main
   ```

Understanding the Commands

==========

1. git remote add upstream

This is used to add another remote repository as a reference, typically when you fork a repository.

- ✅ **When to use:**
  - When you've forked a repository and you want to keep your fork updated with the original repository.

Example:

1. Add the upstream remote:
   ```
   git remote add upstream https://github.com/original-owner/repo.git
   ```
2. Check if upstream is added:
   ```
   git remote -v
   ```

👉 This will show:

```
origin   https://github.com/your-username/repo.git (fetch)
origin   https://github.com/your-username/repo.git (push)
upstream https://github.com/original-owner/repo.git (fetch)
upstream https://github.com/original-owner/repo.git (push)
```

2. git commit

This is used to save your changes locally **(but not on GitHub yet).**

- ✅ **When to use:**
  - After modifying files and staging them using `git add`.

Example:

1. Stage the files:
   `git add .`
2. Create a commit:
   `git commit -m "Add feature X"`

👉 This saves changes locally — you need to `push` to update GitHub.

3. git push

This sends your local commits to the remote repository.

- ✅ **When to use:**
  - After committing changes and wanting to update GitHub.

Example:

Push to the `main` branch:

```
git push origin main
```

Best Workflow for GitHub Collaboration

==========

1. **Clone the repository**
   `git clone https://github.com/your-username/repo.git`

2. **Add the upstream (if needed)**
   `git remote add upstream https://github.com/original-owner/repo.git`

3. **Create a feature branch**
   `git checkout -b feature/branch-name`

4. **Make changes → Stage changes**
   `git add .`
   `git commit -m "Describe your changes"`

5. **Rebase to stay updated**
   `git fetch upstream`
   `git rebase upstream/main`

6. **Push your changes**
   `git push origin feature/branch-name`

7. **Create a pull request** on GitHub
8. **After merging → Update your local branch**
   `git checkout main`
   `git pull --rebase upstream main`

Setup and Best Practices for Your Repository

==========

1. Clone the Repository

If you haven't already cloned the repo:

```
git clone https://github.com/juntabiosGit/appDevWednesday.git
```

Go into the project folder:

```
cd appDevWednesday
```

---

2. Add an Upstream (if it's a fork)

If this repo is a fork from another repository, you need to add an `upstream` to keep it updated with the original source:

```
git remote add upstream https://github.com/original-owner/repo.git
```

👉 Check if it's added correctly:

```
git remote -v
```

You should see:

```
origin   https://github.com/juntabiosGit/appDevWednesday.git (fetch)
origin   https://github.com/juntabiosGit/appDevWednesday.git (push)
upstream https://github.com/original-owner/repo.git (fetch)
upstream https://github.com/original-owner/repo.git (push)
```

---

3. Create a New Branch (Avoid working directly on `main`)

Create a feature branch for your changes:

```
git checkout -b feature/new-feature
```

👉 Example:

- `feature/add-login`
- `fix/bug-in-calculation`

---

4. Make Changes and Commit

1. Stage the changes:
   ```
   git add .
   ```
2. Commit the changes:
   ```
   git commit -m "Add new feature: login"
   ```

---

5. Keep Branch Updated with the Latest Code

Before pushing to GitHub, pull the latest changes using `rebase` to avoid merge conflicts:

```
git fetch origin
git rebase origin/main
```

👉 If conflicts appear:

1. Resolve conflicts manually.
2. After fixing conflicts:
   ```
   git add .
   git rebase --continue
   ```

---

6. Push to GitHub
Push your branch to the remote repository:

```
git push origin feature/new-feature
```

---

7. Create a Pull Request (PR)
1. Go to **https://github.com/juntabiosGit/appDevWednesday**
2. Click **"Compare & pull request"**
3. Write a clear description of changes → Submit PR

Common Issues and Fixes
==========

1. Push rejected due to remote changes
Cause: Someone else pushed changes to `main` before you did.
Solution:
1. First, fetch the latest changes:
   ```
   git fetch origin
   git rebase origin/main
   ```
2. Fix conflicts if necessary → Stage and continue:
   ```
   git add .
   git rebase --continue
   ```
3. Push again:
   ```
   git push origin feature/new-feature
   ```

2. Merge conflicts during pull
Cause: Conflict between local and remote changes.
Solution:
1. Stash your changes:
   ```
   git stash
   ```
2. Pull the latest changes:
   ```
   git pull --rebase origin main
   ```
3. Apply your stashed changes:
   ```
   git stash pop
   ```
4. Fix conflicts → Continue:
   ```
   git add .
   git rebase --continue
   ```

3. Error: “Updates were rejected” after push
Cause: Someone updated the remote branch after you committed.
Solution:
1. Rebase to update:
   ```
   git pull --rebase origin main
   ```
2. Fix conflicts if needed → Continue:
   ```
   git add .
   git rebase --continue
   ```
3. Push:
   ```
   git push origin feature/new-feature
   ```

Git and GitHub Commands Cheat Sheet

==========

1. Basic Git Commands

```
git init - Initialize a new Git repository in a folder
git clone <repo_url> - Clone a remote repository to your local machine
git status - Check the status of your repository (modified, staged,
untracked files)
git add <file> - Stage a specific file for commit
git add -A - Stage all modified and new files
git commit -m "message" - Commit staged files with a message
git log - View commit history
git diff - Show changes between commits, branches, or files
```

2. Branching and Merging

```
git branch - List all branches
git branch <branch_name> - Create a new branch
git checkout <branch_name> - Switch to another branch
git switch <branch_name> - Alternative to checkout for switching
branches
git merge <branch_name> - Merge a branch into the current branch
git branch -d <branch_name> - Delete a branch (if merged)
git branch -D <branch_name> - Force delete a branch
```

3. Working with Remote Repositories (GitHub)

```
git remote -v - Show remote repositories linked to the local repo
git remote add origin <repo_url> - Add a remote repository
git push origin <branch_name> - Push changes to GitHub
git push -u origin <branch_name> - Push and track a new branch to
GitHub
git pull origin <branch_name> - Pull the latest changes from GitHub
git fetch - Fetch updates from the remote repository without merging
```

4. Undo Changes

```
git reset --hard - Reset all changes (warning: cannot be undone)
git reset --soft HEAD~1 - Undo last commit but keep changes staged
git reset HEAD <file> - Unstage a file

git checkout -- <file> - Discard local changes in a file
```

5. Deleting Files & Repositories

```
rm <file> - Delete a file locally
rm -r <folder> - Delete a folder locally
git rm <file> - Remove a file from Git and stage the deletion
git commit -m "Deleted file" - Commit the deletion
git push origin main - Push the deletion to GitHub
To delete a repository on GitHub: Go to Settings > Danger Zone >
Delete this repository
```

6. GitHub Collaboration Commands

```
git fork - Copy someone else's GitHub repo to your account
git clone <forked_repo_url> - Clone a forked repository
git remote add upstream <original_repo_url> - Link your fork to the
original repository
git fetch upstream - Fetch updates from the original repository
git merge upstream/main - Merge the latest changes from the original
repo
```

Complete Git Commands Reference

==========

Command	Description
git init	Initialize a new Git repository.
git clone <repo-url>	Clone a remote repository to your local machine.
git add .	Stage all changes for the next commit.
git add <file>	Stage a specific file for the next commit.
git commit -m "message"	Create a commit with a message describing the changes.
git status	Show the status of changes (staged, unstaged, etc.).
git log	View commit history.
git log --oneline	View a simplified commit history.
git branch	List all branches.

git branch <branch-name>	Create a new branch.
git checkout <branch-name>	Switch to a specific branch.
git checkout -b <branch-name>	Create and switch to a new branch.
git merge <branch-name>	Merge another branch into the current branch.
git rebase <branch-name>	Reapply commits on top of another branch.
git fetch	Download changes from the remote repository without merging.
git pull	Fetch and merge changes from the remote repository.
git pull --rebase	Fetch and reapply commits on top of the current branch.
git push origin <branch-name>	Push changes to the remote repository.
git push -u origin <branch-name>	Push changes and set upstream for future pushes.
git remote add origin <url>	Link a local repository to a remote repository.
git remote -v	Show the remote repository URL.
git stash	Temporarily save changes without committing.
git stash pop	Reapply stashed changes.
git stash list	List all stashed changes.
git stash drop	Remove a specific stash.
git diff	Show differences between commits, branches, and the working directory.
git diff --staged	Show differences between staged files and the last commit.
git reset --soft <commit>	Reset to a specific commit, keeping changes staged.
git reset --mixed <commit>	Reset to a specific commit, keeping changes unstaged.

git reset --hard <commit>	Reset to a specific commit, discarding all changes.
git rm <file>	Remove files from the working directory and stage the deletion.
git mv <old-name> <new-name>	Rename a file and stage the change.
git revert <commit>	Create a new commit that undoes a specific commit.
git clean -f	Remove untracked files from the working directory.
git tag <tag-name>	Create a new tag at the current commit.
git tag	List all tags.
git tag -d <tag-name>	Delete a specific tag.
git cherry-pick <commit>	Apply a specific commit from another branch.
git bisect start	Start a binary search to find a bug.
git bisect good <commit>	Mark a commit as 'good' during bisect.
git bisect bad <commit>	Mark a commit as 'bad' during bisect.
git bisect reset	End the bisect process.
git show <commit>	Show details of a specific commit.
git blame <file>	Show who last modified each line in a file.
git shortlog	Summarize commit history by author.
git reflog	Show a log of changes to the branch history.
git config --global user.name "Your Name"	Set the global username for Git.
git config --global user.email "your-email@example.com"	Set the global email for Git.
git config --list	List all Git configuration settings.
git config --global core.editor "code"	Set the default Git editor to VS Code.

git merge --abort	Abort a merge in progress.
git cherry-pick -n <commit>	Apply a commit without creating a new commit.
git rm --cached <file>	Unstage a file without deleting it.
git diff <branch>..<branch>	Show differences between two branches.
git ls-files	List files in the working directory.
git cat-file -p <hash>	Show the content of a specific commit object.
git archive -o <archive>.zip HEAD	Create a zip file of the current branch.
git fsck	Check the integrity of the repository.
git gc	Clean up unnecessary files and optimize the repository.
git prune	Remove unreachable objects from the repository.
git whatchanged	Show changes to the repository history.
git citool	Launch a GUI tool to review commits.
gitk	Launch a GUI to visualize the commit history.
git worktree add <path> <branch>	Create a new working tree from a branch.
git worktree list	List all working trees.
git worktree remove <path>	Remove a working tree.
git grep <pattern>	Search for a pattern in the repository files.
git fsck --full	Perform a full integrity check on the repository.
git log --graph	Show commit history as a graph.
git log --since="2 weeks ago"	Show commits from the last two weeks.
git blame -L 5,10 <file>	Show who modified lines 5 to 10 of a file.
git checkout -- <file>	Discard local changes to a specif

🔑 Golden Rules to Avoid Sync Issues

✔ Always pull using `--rebase` before making changes.
✔ Push changes only when you've confirmed it won't break the code.
✔ Use feature branches instead of working directly on `main`.
✔ Commit frequently and write clear commit messages.
✔️ Use `git stash` to temporarily store changes if you encounter conflicts.

🚀 Ideal Workflow

1. **Start working** → `git pull --rebase origin main`
2. **Create a feature branch** → `git checkout -b feature/branch-name`
3. **Make changes** → `git add .` → `git commit -m "Message"`
4. **Sync latest code** → `git pull --rebase origin main`
5. **Push changes** → `git push origin feature/branch-name`
6. **Create a pull request** → Once reviewed → Merge to `main`